

## 0: Unzipper

```
1 !unzip "/content/drive/MyDrive/Colab Notebooks/images.zip" -d "/content/"
```

[Show hidden output](#)

## 1: Octupler (Offline Augmentation)

```
1 import os
2 import shutil
3 import random
4 from PIL import Image, ImageOps
5 from tqdm.notebook import tqdm
6
7 # =====
8 # CONFIGURATION
9 # =====
10 INPUT_DIR = '/content/images' # Where your original folders are
11 OUTPUT_DIR = 'dataset_augmented_8x' # New Massive Folder
12
13 # Clean start
14 if os.path.exists(OUTPUT_DIR):
15     shutil.rmtree(OUTPUT_DIR)
16
17 # Create Structure
18 os.makedirs(os.path.join(OUTPUT_DIR, 'train'), exist_ok=True)
19 os.makedirs(os.path.join(OUTPUT_DIR, 'validation'), exist_ok=True)
20
21 print(f"🚀 STARTING 8x DATASET EXPANSION...")
22 print(f"   Source: {INPUT_DIR}")
23 print(f"   Target: {OUTPUT_DIR}")
24
25 # Get class names
26 classes = [d for d in os.listdir(INPUT_DIR) if os.path.isdir(os.path.join(INPUT_DIR, d))]
27 classes = [c for c in classes if not c.startswith('.')] # Skip hidden files
28
29 for class_name in classes:
30     print(f"\nProcessing Class: {class_name}...")
31
32     # Create subfolders
33     os.makedirs(os.path.join(OUTPUT_DIR, 'train', class_name), exist_ok=True)
34     os.makedirs(os.path.join(OUTPUT_DIR, 'validation', class_name), exist_ok=True)
35
36     # Get Images
37     src_path = os.path.join(INPUT_DIR, class_name)
38     images = [f for f in os.listdir(src_path) if f.lower().endswith(('.jpg', '.jpeg', '.png', '.bmp', '.webp'))]
39
40     # 1. SPLIT FIRST (Crucial for Integrity)
41     random.seed(42)
42     random.shuffle(images)
43
44     split_idx = int(len(images) * 0.8)
45     train_imgs = images[:split_idx]
46     val_imgs = images[split_idx:]
47
48     # 2. HANDLE VALIDATION (Pure Copy)
49     print(f"   - Moving {len(val_imgs)} images to Validation (Untouched)")
50     for img_name in val_imgs:
51         shutil.copy(
52             os.path.join(src_path, img_name),
53             os.path.join(OUTPUT_DIR, 'validation', class_name, img_name)
54         )
55
56     # 3. HANDLE TRAINING (The 8x Generator)
57     print(f"   - Generating {len(train_imgs) * 8} images for Training...")
58
59     for img_name in tqdm(train_imgs, leave=False):
60         full_path = os.path.join(src_path, img_name)
61
62         try:
63             with Image.open(full_path) as img:
64                 img = img.convert('RGB')
65
```

```

66         # We will generate 4 rotations, and for each rotation, we also save a flipped version.
67         # Angles: 0, 90, 180, 270
68
69         angles = [0, 90, 180, 270]
70
71         for angle in angles:
72             # 1. Rotate
73             if angle == 0:
74                 rot_img = img
75             else:
76                 rot_img = img.rotate(angle, expand=True)
77
78             # Save Rotated
79             rot_name = f"rot{angle}_{img_name}"
80             rot_img.save(os.path.join(OUTPUT_DIR, 'train', class_name, rot_name))
81
82             # 2. Flip the Rotated version
83             flip_img = ImageOps.mirror(rot_img)
84
85             # Save Flipped
86             flip_name = f"flip_rot{angle}_{img_name}"
87             flip_img.save(os.path.join(OUTPUT_DIR, 'train', class_name, flip_name))
88
89         except Exception as e:
90             print(f"⚠️ Error processing {img_name}: {e}")
91
92 print(f"\n✅ DONE! Dataset expanded 8x in '{OUTPUT_DIR}')
```

🚀 STARTING 8x DATASET EXPANSION...

Source: /content/images

Target: dataset\_augmented\_8x

Processing Class: Bacterial-Wilt...

- Moving 32 images to Validation (Untouched)
- Generating 1024 images for Training...

Processing Class: Pythium-Fruit-Rot...

- Moving 32 images to Validation (Untouched)
- Generating 1024 images for Training...

Processing Class: Anthracnose...

- Moving 32 images to Validation (Untouched)
- Generating 1024 images for Training...

Processing Class: Fresh-Cucumber...

- Moving 32 images to Validation (Untouched)
- Generating 1024 images for Training...

Processing Class: Downy-Mildew...

- Moving 32 images to Validation (Untouched)
- Generating 1024 images for Training...

Processing Class: Belly-Rot...

- Moving 32 images to Validation (Untouched)
- Generating 1024 images for Training...

Processing Class: Gummy-Stem-Blight...

- Moving 32 images to Validation (Untouched)
- Generating 1024 images for Training...

Processing Class: Fresh-Leaf...

- Moving 32 images to Validation (Untouched)
- Generating 1024 images for Training...

✅ DONE! Dataset expanded 8x in 'dataset\_augmented\_8x'

## 2: Training the Attention Model on New Data

```

1 import tensorflow as tf
2 from tensorflow.keras import layers, models, optimizers, callbacks
```

```

2 from tensorflow.keras import layers, models, optimizers, callbacks
3 from tensorflow.keras.preprocessing.image import ImageDataGenerator
4 from sklearn.utils import class_weight
5 import numpy as np
6
7 # =====
8 # 1. SETUP (New Path)
9 # =====
10 DATA_PATH = 'dataset_augmented_8x' # <--- NEW 8x FOLDER
11 IMG_SIZE = (256, 256)
12 BATCH_SIZE = 32
13
14 # Since we already rotated AND flipped manually, we turn OFF the generator's geometry augs.
15 # We only keep Shift/Zoom for slight "wobble".
16 train_datagen = ImageDataGenerator(
17     rescale=1.0/255.0,
18     width_shift_range=0.1,
19     height_shift_range=0.1,
20     zoom_range=0.1,
21     fill_mode='nearest'
22     # Note: No rotation_range, No horizontal_flip (already done offline!)
23 )
24
25 val_datagen = ImageDataGenerator(rescale=1.0/255.0)
26
27 print("--- Loading 8x Data ---")
28 train_generator = train_datagen.flow_from_directory(
29     os.path.join(DATA_PATH, 'train'),
30     target_size=IMG_SIZE,
31     batch_size=BATCH_SIZE,
32     class_mode='categorical'
33 )
34
35 validation_generator = val_datagen.flow_from_directory(
36     os.path.join(DATA_PATH, 'validation'),
37     target_size=IMG_SIZE,
38     batch_size=BATCH_SIZE,
39     class_mode='categorical',
40     shuffle=False
41 )
42
43 # =====
44 # 2. DEFINE ATTENTION MODEL (Same as before)
45 # =====
46 def squeeze_excite_block(input_tensor, ratio=16):
47     init = input_tensor
48     filters = init.shape[-1]
49     se = layers.GlobalAveragePooling2D()(init)
50     se = layers.Dense(filters // ratio, activation='relu', use_bias=False)(se)
51     se = layers.Dense(filters, activation='sigmoid', use_bias=False)(se)
52     return layers.Multiply()([init, se])
53
54 def build_attention_model(num_classes):
55     inputs = layers.Input(shape=(256, 256, 3))
56
57     # Block 1
58     x = layers.Conv2D(32, (3, 3), padding='same', kernel_initializer='he_normal')(inputs)
59     x = layers.BatchNormalization()(x)
60     x = layers.Activation('relu')(x)
61     x = squeeze_excite_block(x)
62     x = layers.MaxPooling2D((2, 2))(x)
63
64     # Block 2
65     x = layers.Conv2D(64, (3, 3), padding='same', kernel_initializer='he_normal')(x)
66     x = layers.BatchNormalization()(x)
67     x = layers.Activation('relu')(x)
68     x = squeeze_excite_block(x)
69     x = layers.MaxPooling2D((2, 2))(x)
70
71     # Block 3
72     x = layers.Conv2D(128, (3, 3), padding='same', kernel_initializer='he_normal')(x)
73     x = layers.BatchNormalization()(x)
74     x = layers.Activation('relu')(x)
75     x = squeeze_excite_block(x)
76     x = layers.MaxPooling2D((2, 2))(x)
77
78     # Block 4
79     x = layers.Conv2D(256, (3, 3), padding='same', kernel_initializer='he_normal')(x)
80     x = layers.BatchNormalization()(x)
81     x = layers.Activation('relu')(x)
82     x = layers.MaxPooling2D((2, 2))(x)
83
84     # Block 5

```

```

85     x = layers.Conv2D(512, (3, 3), padding='same', kernel_initializer='he_normal')(x)
86     x = layers.BatchNormalization()(x)
87     x = layers.Activation('relu')(x)
88     x = layers.MaxPooling2D((2, 2))(x)
89
90     # Classifier
91     avg_pool = layers.GlobalAveragePooling2D()(x)
92     max_pool = layers.GlobalMaxPooling2D()(x)
93     x = layers.Concatenate()([avg_pool, max_pool])
94
95     x = layers.Dense(512, activation='relu', kernel_initializer='he_normal')(x)
96     x = layers.BatchNormalization()(x)
97     x = layers.Dropout(0.5)(x)
98
99     outputs = layers.Dense(num_classes, activation='softmax')(x)
100     return models.Model(inputs, outputs)
101
102 model = build_attention_model(train_generator.num_classes)
103
104 # =====
105 # 3. COMPILE & TRAIN
106 # =====
107 optimizer = optimizers.Adam(learning_rate=0.0005)
108 loss_fn = tf.keras.losses.CategoricalCrossentropy(label_smoothing=0.1)
109
110 model.compile(optimizer=optimizer, loss=loss_fn, metrics=['accuracy'])
111
112 checkpoint = callbacks.ModelCheckpoint('best_8x_model.keras',
113                                     monitor='val_accuracy', save_best_only=True, mode='max')
114 # Reduced patience because 8x data implies many steps per epoch
115 reduce_lr = callbacks.ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=2, min_lr=1e-6)
116 early_stop = callbacks.EarlyStopping(monitor='val_loss', patience=6, restore_best_weights=True)
117
118 # Weights
119 class_weights = class_weight.compute_class_weight(
120     class_weight='balanced',
121     classes=np.unique(train_generator.classes),
122     y=train_generator.classes
123 )
124 class_weight_dict = dict(enumerate(class_weights))
125
126 print("\n🚀 TRAINING ON 8x DATASET...")
127 history = model.fit(
128     train_generator,
129     epochs=35,
130     validation_data=validation_generator,
131     callbacks=[checkpoint, reduce_lr, early_stop],
132     class_weight=class_weight_dict
133 )

```

--- Loading 8x Data ---

Found 8192 images belonging to 8 classes.

Found 256 images belonging to 8 classes.

🚀 TRAINING ON 8x DATASET...

/usr/local/lib/python3.12/dist-packages/keras/src/trainers/data\_adapters/py\_dataset\_adapter.py:121: UserWarning: Your `PyDatasetAdapter` class is not a subclass of `PyDatasetAdapter`.  
self.\_warn\_if\_super\_not\_called()

Epoch 1/35

256/256 ————— 306s 1s/step - accuracy: 0.6068 - loss: 1.4565 - val\_accuracy: 0.2500 - val\_loss: 3.1497 - learning\_rate: 0.0005

Epoch 2/35

256/256 ————— 295s 1s/step - accuracy: 0.8120 - loss: 0.9497 - val\_accuracy: 0.7422 - val\_loss: 1.1352 - learning\_rate: 0.0005

Epoch 3/35

256/256 ————— 287s 1s/step - accuracy: 0.8697 - loss: 0.8193 - val\_accuracy: 0.7930 - val\_loss: 1.0285 - learning\_rate: 0.0005

Epoch 4/35

218/256 ————— 40s 1s/step - accuracy: 0.9154 - loss: 0.7300 - val\_accuracy: 0.8120 - val\_loss: 1.1352 - learning\_rate: 0.0005

## 3: Evaluate with TTA

```

1 from sklearn.metrics import f1_score, classification_report
2 import numpy as np
3
4 print("\n🚀 STARTING TTA EVALUATION (8x Model)...")
5 model.load_weights('best_8x_model.keras')
6
7 # TTA Generator (Flip only)
8 tta_datagen = ImageDataGenerator(rescale=1.0/255.0, horizontal_flip=True)
9
10 # 1. Standard Prediction
11 print("📊 Standard Pass...")
12 validation_generator.reset()

```

```

13 preds_standard = model.predict(validation_generator, verbose=1)
14
15 # 2. Flipped Prediction
16 print("2 Flipped Pass...")
17 flipped_generator = tta_datagen.flow_from_directory(
18     os.path.join(DATA_PATH, 'validation'),
19     target_size=(256, 256),
20     batch_size=BATCH_SIZE,
21     class_mode='categorical',
22     shuffle=False
23 )
24 preds_flipped = model.predict(flipped_generator, verbose=1)
25
26 # 3. Average
27 final_preds = (0.5 * preds_standard) + (0.5 * preds_flipped)
28 predicted_classes = np.argmax(final_preds, axis=1)
29 true_classes = validation_generator.classes
30
31 # 4. Score
32 f1 = f1_score(true_classes, predicted_classes, average='weighted')
33 print(f"\n2 FINAL F1 SCORE (8x Data + TTA): {f1:.5f}")

```

🚀 STARTING TTA EVALUATION...

1 Standard Pass...

8/8 ██████████ 6s 536ms/step

2 Flipped Pass...

Found 256 images belonging to 8 classes.

8/8 ██████████ 5s 607ms/step

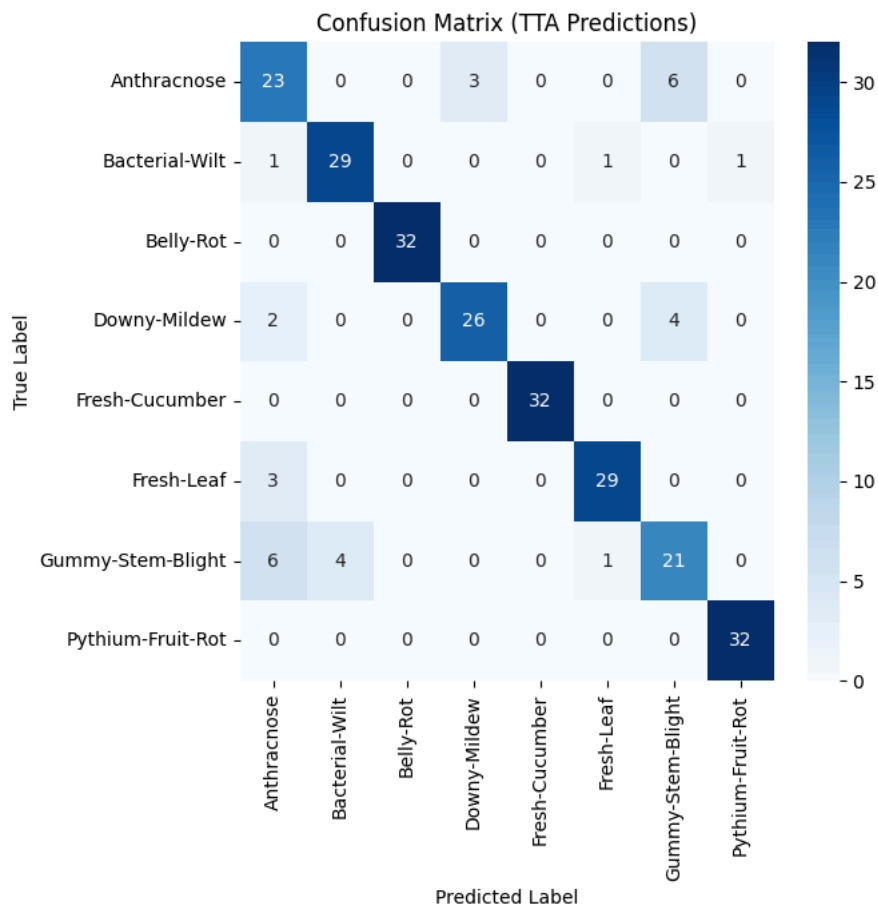
2 FINAL F1 SCORE (Augmented + TTA): 0.87541

## Confusion Matrix

```

1 import matplotlib.pyplot as plt
2 import seaborn as sns
3 from sklearn.metrics import confusion_matrix
4
5 cm = confusion_matrix(true_classes, predicted_classes)
6
7 # Class names from generator
8 class_labels = list(validation_generator.class_indices.keys())
9
10 plt.figure(figsize=(7, 7))
11 sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
12             xticklabels=class_labels,
13             yticklabels=class_labels)
14
15 plt.title("Confusion Matrix (TTA Predictions)")
16 plt.xlabel("Predicted Label")
17 plt.ylabel("True Label")
18 plt.tight_layout()
19 plt.show()
20

```



## Graphs

```

1 import matplotlib.pyplot as plt
2
3 # ACCURACY
4 plt.figure(figsize=(4, 3))
5 plt.plot(history.history['accuracy'], label='Training Accuracy')
6 plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
7 plt.title('Training vs Validation Accuracy')
8 plt.xlabel('Epochs')
9 plt.ylabel('Accuracy')
10 plt.legend()
11 plt.grid(True)
12 plt.tight_layout()
13 plt.show()
14
15 # LOSS
16 plt.figure(figsize=(4, 3))
17 plt.plot(history.history['loss'], label='Training Loss')
18 plt.plot(history.history['val_loss'], label='Validation Loss')
19 plt.title('Training vs Validation Loss')
20 plt.xlabel('Epochs')
21 plt.ylabel('Loss')
22 plt.legend()
23 plt.grid(True)
24 plt.tight_layout()
25 plt.show()
26
27

```

